

P3257R0: Make the predicate of `contract_assert` more regular

Introduction

Discussions in EWG and SG21 and on the respective reflectors have shown that the changes effected by P3071R0 (Protection against modifications in contracts) cause concerns, because different behavior for contract predicates compared to nearby code is established.

This paper proposes (1) to revert P3071R0 for the interpretation of predicates in `contract_assert` and (2) to specify exactly-once evaluation of predicates in `contract_assert` unless the contract has the "ignore" semantic.

History

- R0: Initial paper.

Discussion

Const view in `contract_assert`

In general, contract predicates should check the state of the program, but not modify it. P3071R0 strives to aid with that by making accesses to local variables and data members const-qualified, the latter akin to const member functions. It is well understood that this may change overload resolution results or make certain expressions ill-formed, for example `map[key]`.

This approach makes the interpretation of a given expression different inside and outside of e.g. a `contract_assert`:

```
void f() {  
    int i = 0;  
    if (++i < 5) { ... }           // OK  
    contract_assert(++i < 5);     // P2900R6: ill-formed; proposed: well-formed  
}
```

Furthermore, some interfaces even in the standard library do not offer const overloads, e.g. `map[key]` or `std::ios_base::iword`:

```
int g(std::ios_base& io) {  
    int idx = io.xalloc();  
    if (io.iword(idx)) { ... }    // OK  
    contract_assert(io.iword(idx) == 0); // P2900R6: ill-formed; proposed: well-formed  
    return idx;  
}
```

This is contrary to popular expectations that the same source code appearing twice in lexical vicinity works the same.

This argument applies to a lesser extent to preconditions and postconditions, because those appear in the declaration, possibly far away from the definition:

```
void h(int x, int y)
  pre(x > 0)           // ok
  pre(y < 0)           // ok
  pre(++x < 42);      // ill-formed
```

Thus, as "proposal 1", this paper suggests to revert P3071R0 for the interpretation of predicates in `contract_assert` (but not in preconditions or postconditions).

Evaluation semantics

Code patterns involving C-style `assert` have been discovered that intentionally modify state only to be used for checking:

```
void f() {
#ifdef NDEBUG
    int iter = 0;
#endif
    while (/* something */) {
        assert(++iter < 6);    // iterating more than five times is a bug
        // ...
    }
}
```

Beyond the fact that there is currently no way in P2900R6 to appropriately guard the declaration of the variable `iter`, there is also no guarantee that this code works as expected to start with when transforming the `assert` into a `contract_assert`, because contract predicates are allowed to be evaluated an indefinite number of times, per P2900R6:

```
void f() {
    int iter = 0;
    while (/* something */) {
        contract_assert(++iter < 6);    // well-formed with "proposal 1"
        // ...
    }
}
```

There are engineering reasons why preconditions and postconditions need the liberty to be evaluated up to twice, in order to support separate compilation and delivery of callers and callees, but those reasons do not apply to `contract_assert`, which always appears inside a function body (or lambda). The allowance to evaluate more than twice was introduced to discourage contract predicates that change state, because the resulting new state would be unpredictable. For `contract_assert`, this goal seems less important than the ability to concisely implement the use-cases for state modification shown above.

For another example:

```
if (!expr) {    // #1
    // modify the state to make `expr` true
    ...
}
contract_assert(expr);    // P2900R6: semantics of "expr" not guaranteed to match #1
```

Thus, as "proposal 2", this paper suggests to guarantee exactly-once evaluation of predicates in `contract_assert`, unless the contract has the "ignore" semantic (in which case no evaluation happens).

Outline of wording changes relative to P2900R6

Proposal 1: Interpretation of predicates in `contract_assert`

- Adjust the note in [basic.contract.general] p3.
- Limit the new paragraph in [expr.prim.this] to preconditions and postconditions.
- Limit [expr.prim.id.unqual] p5 to preconditions and postconditions.

Proposal 2: Exactly-once evaluation of predicates in `contract_assert`

- Limit [basic.contract.eval] p6 to preconditions and postconditions.

Acknowledgements

Thanks to Gabriel Dos Reis for reviewing a draft of this paper.